

NCTU-EE IC LAB – Fall 2018

Midterm Exam 2018/11/10 10:00~12:00

1. Alice wrote a Verilog program but cannot run. There are 5 **syntax errors** in following code, please find and correct them. (10%)

```
module test();
    reg out, a, b, select, clk, rst;
    mux2_1 (.out(out), .a(a), .b(b), .select(select), .clk(clk), .rst(rst) );
    always #(1) clk = ~clk;
    initial begin
        clk = 0; rst = 1;
        #(5) rst = 0;
        a = 0; b = 1; select = 0;
        #(5) select = 1;
        #(5) $finish;
    end
endmodule

module mux2_1(out, a, b, select, clk, rst);
    input reg a, b, select, clk, rst;
    output wire out;
    wire 2_to_1;
    assign 2_to_1 = (select == 1'b0) ? a : b;
    always@(posedge clk or posedge rst)
    begin
        if(rst == 1'b1) out <= 1'b0;
        else out <= 2_to_1;
    end
endmodule
```

2. Peter is trying to write a combination logic, which $x=a \& b$ and $y=(a \& b) | c | d$; He wrote the program as follows but get the wrong rtl simulation result.

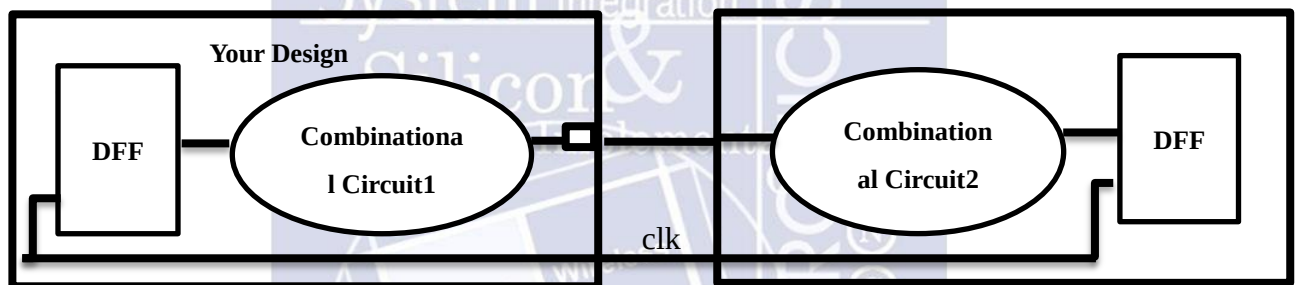
```
always@(a or b or c or d) begin
    y = x | c | d;
    x = a & b;
end
```

- (a) Please explain why the rtl result is wrong. (4%)
- (b) Replace the sensitive list by always @(*) will make it correct, explain what always @(*) means and does in this case. (3%)
- (c) Use another way to make the rtl simulation correct other than changing the sensitive list. (3%)
3. In previous labs, we are performing the front-end progress of the cell based design flow, which are rtl simulation, synthesis, and gate level simulation.
- (a) Please describe the advantages and disadvantages of the cell-based design flow (compared to full custom design flow, try to list as more as you can) (5%)
- (b) Please describe in these three stages, what input files should be prepared and what output results / files will be generated? (10%)
4. Following is a table of different timescale settings with different delay declarations. Please calculate the actual values we will get in our pattern file with simulation. (10%)
- | | `timescale Unit/Precision | Delay Declaration | Actual Delay |
|-----|----------------------------------|--------------------------|---------------------|
| (a) | `timescale 10ns/1ns | #5 | ? |
| (b) | `timescale 10ns/1ns | #5.738 | ? |
| (c) | `timescale 10ns/10ns | #5.1 | ? |
| (d) | `timescale 10ns/100ps | #5.732 | ? |
5. Wendy has written an RTL code and synthesized it without latch and error. She also checked the synthesis cycle is the same as pattern. However, she forgot to use \$sdf_annotate() task in the TESTBED in gate level simulation, and she got many “Warning! Timing violation ...” messages. Please explain why. (5%)
6. (a) Please describe why we need to specify input delay and output delay. What happened if we set input delay or output delay larger? (5%)
- (b) Give the formulas of setup time timing constraints of
- reg to reg (4%)
 - input to reg (3%)
 - reg to output (3%)
- in terms of (T_{cycle} : clock cycle, t_i : input delay, t_o : output delay, t_{cq} : clk-to-q propagation delay, t_d : combinational circuit propagation delay, t_s : setup time)

7. Sam is asked to design a circuit to communicate with others. The ports are going to connect to another design as shown in the figure. The other design has already been synthesized; and we assume the timing properties of the DFF is fixed, as shown in the following. What is the minimum value of the output delay you could set in your design's .tcl file. (5%)

$T_{\text{combinational Circuit 2}}$: 20ns
 T_{cq} : 2ns.
 T_{setup} : 5ns
 T_{hold} : 3ns

dc_shell> set_output_delay – clock clk _____ [all_outputs]



8. When calling designware or memory, we use commands:
`//synopsys translate_off`
``include "xxx.v"`
`//synopsys translate_on`
 Explain what `//synopsys translate_off` and `//synopsys translate_on` mean and why we use those commands. (In rtl simulation and synthesis view) (5%)
9. List three major kinds of IP and describe them. (10%)
10. When using memory, we use a behavior netlist which cannot be synthesized to simulate the memory behavior in rtl simulation. How about in gate level simulation? What kind of memory file do we use to simulate? (5%)

11. Please read the following netlist and fill the corresponding optimization method.
(uniquify (4%), set_dont_touch (3%), ungroup (3%))

Original:

```
module ch6_ex(input clk, INF.DESIGN inf);
    adder_1b add1(.clk(clk),.rst_n(inf.rst_n),.x1(inf.x1),.x2(inf.x2),.Sum(inf.Sum1));
    adder_1b add2(.clk(clk),.rst_n(inf.rst_n),.x1(inf.x1),.x2(inf.x2),.Sum(inf.Sum2));
    adder_1b add3(.clk(clk),.rst_n(inf.rst_n),.x1(inf.x1),.x2(inf.x2),.Sum(inf.Sum3));
endmodule :ch6_ex

module adder_1b ( clk, rst_n, x1, x2, Sum );
    output [1:0] Sum;
    input clk, rst_n, x1, x2;
    wire N1, N2;

    DFFTRX1 Sum_reg_1 ( .D(N2), .RN(rst_n), .CK(clk), .Q(Sum[1]) );
    DFFTRX1 Sum_reg_0 ( .D(N1), .RN(rst_n), .CK(clk), .Q(Sum[0]) );
    XOR2X2 U3 ( .A(x2), .B(x1), .Y(N1) );
    AND2X2 U4 ( .A(x1), .B(x2), .Y(N2) );
endmodule
```

(a)

```
module ch6_ex ( clk, inf_rst_n, inf_x1, inf_x2, inf_Sum1, inf_Sum2, inf_Sum3
);
    output [1:0] inf_Sum1;
    output [1:0] inf_Sum2;
    output [1:0] inf_Sum3;
    input clk, inf_rst_n, inf_x1, inf_x2;

    adder_1b add1 ( .clk(clk), .rst_n(inf_rst_n), .x1(inf_x1), .x2(inf_x2),
        .Sum(inf_Sum1) );
    adder_1b add2 ( .clk(clk), .rst_n(inf_rst_n), .x1(inf_x1), .x2(inf_x2),
        .Sum(inf_Sum2) );
    adder_1b_0 add3 ( .clk(clk), .rst_n(inf_rst_n), .x1(inf_x1), .x2(inf_x2),
        .Sum(inf_Sum3) );
endmodule

module adder_1b_0 ( clk, rst_n, x1, x2, Sum );
    output [1:0] Sum;
    input clk, rst_n, x1, x2;
    wire N2, N1;

    DFFTRX1 Sum_reg_1 ( .D(N2), .RN(rst_n), .CK(clk), .Q(Sum[1]) );
    DFFTRX1 Sum_reg_0 ( .D(N1), .RN(rst_n), .CK(clk), .Q(Sum[0]) );
    XOR2X1 U1 ( .A(x2), .B(x1), .Y(N1) );
    AND2X1 U2 ( .A(x1), .B(x2), .Y(N2) );
endmodule

module adder_1b ( clk, rst_n, x1, x2, Sum );
    output [1:0] Sum;
    input clk, rst_n, x1, x2;
    wire N2, N1;

    DFFTRX1 Sum_reg_1 ( .D(N2), .RN(rst_n), .CK(clk), .Q(Sum[1]) );
    DFFTRX1 Sum_reg_0 ( .D(N1), .RN(rst_n), .CK(clk), .Q(Sum[0]) );
    XOR2X2 U3 ( .A(x2), .B(x1), .Y(N1) );
    AND2X2 U4 ( .A(x1), .B(x2), .Y(N2) );
endmodule
```

(c)

```
module ch6_ex ( clk, inf_rst_n, inf_x1, inf_x2, inf_Sum1, inf_Sum2, inf_Sum3
);
    output [1:0] inf_Sum1;
    output [1:0] inf_Sum2;
    output [1:0] inf_Sum3;
    input clk, inf_rst_n, inf_x1, inf_x2;
    wire add2_N1, add2_N2;

    adder_1b add3 ( .clk(clk), .rst_n(inf_rst_n), .x1(inf_x1), .x2(inf_x2),
        .Sum(inf_Sum3) );
    DFFTRX1 add1_Sum_reg_0 ( .D(add2_N1), .RN(inf_rst_n), .CK(clk), .Q(
        inf_Sum1[0]) );
    DFFTRX1 add1_Sum_reg_1 ( .D(add2_N2), .RN(inf_rst_n), .CK(clk), .Q(
        inf_Sum1[1]) );
    DFFTRX1 add2_Sum_reg_0 ( .D(add2_N1), .RN(inf_rst_n), .CK(clk), .Q(
        inf_Sum2[0]) );
    DFFTRX1 add2_Sum_reg_1 ( .D(add2_N2), .RN(inf_rst_n), .CK(clk), .Q(
        inf_Sum2[1]) );
    XOR2X1 U3 ( .A(inf_x1), .B(inf_x2), .Y(add2_N1) );
    AND2X1 U4 ( .A(inf_x2), .B(inf_x1), .Y(add2_N2) );
endmodule

module adder_1b ( clk, rst_n, x1, x2, Sum );
    output [1:0] Sum;
    input clk, rst_n, x1, x2;
    wire N2, N1;

    DFFTRX1 Sum_reg_1 ( .D(N2), .RN(rst_n), .CK(clk), .Q(Sum[1]) );
    DFFTRX1 Sum_reg_0 ( .D(N1), .RN(rst_n), .CK(clk), .Q(Sum[0]) );
    XOR2X1 U1 ( .A(x2), .B(x1), .Y(N1) );
    AND2X1 U2 ( .A(x1), .B(x2), .Y(N2) );
endmodule
```

(b)

```
module ch6_ex ( clk, inf_rst_n, inf_x1, inf_x2, inf_Sum1, inf_Sum2, inf_Sum3
);
    output [1:0] inf_Sum1;
    output [1:0] inf_Sum2;
    output [1:0] inf_Sum3;
    input clk, inf_rst_n, inf_x1, inf_x2;

    adder_1b_2 add1 ( .clk(clk), .rst_n(inf_rst_n), .x1(inf_x1), .x2(inf_x2),
        .Sum(inf_Sum1) );
    adder_1b_1 add2 ( .clk(clk), .rst_n(inf_rst_n), .x1(inf_x1), .x2(inf_x2),
        .Sum(inf_Sum2) );
    adder_1b_0 add3 ( .clk(clk), .rst_n(inf_rst_n), .x1(inf_x1), .x2(inf_x2),
        .Sum(inf_Sum3) );
endmodule

module adder_1b_1 ( clk, rst_n, x1, x2, Sum );
    output [1:0] Sum;
    input clk, rst_n, x1, x2;
    wire N2, N1;

    DFFTRX1 Sum_reg_1 ( .D(N2), .RN(rst_n), .CK(clk), .Q(Sum[1]) );
    DFFTRX1 Sum_reg_0 ( .D(N1), .RN(rst_n), .CK(clk), .Q(Sum[0]) );
    XOR2X1 U1 ( .A(x2), .B(x1), .Y(N1) );
    AND2X1 U2 ( .A(x1), .B(x2), .Y(N2) );
endmodule

module adder_1b_0 ( clk, rst_n, x1, x2, Sum );
    output [1:0] Sum;
    input clk, rst_n, x1, x2;
    wire N2, N1;

    DFFTRX1 Sum_reg_1 ( .D(N2), .RN(rst_n), .CK(clk), .Q(Sum[1]) );
    DFFTRX1 Sum_reg_0 ( .D(N1), .RN(rst_n), .CK(clk), .Q(Sum[0]) );
    XOR2X1 U1 ( .A(x2), .B(x1), .Y(N1) );
    AND2X1 U2 ( .A(x1), .B(x2), .Y(N2) );
endmodule

module adder_1b_2 ( clk, rst_n, x1, x2, Sum );
    output [1:0] Sum;
    input clk, rst_n, x1, x2;
    wire N2, N1;

    DFFTRX1 Sum_reg_1 ( .D(N2), .RN(rst_n), .CK(clk), .Q(Sum[1]) );
    DFFTRX1 Sum_reg_0 ( .D(N1), .RN(rst_n), .CK(clk), .Q(Sum[0]) );
    XOR2X1 U1 ( .A(x2), .B(x1), .Y(N1) );
    AND2X1 U2 ( .A(x1), .B(x2), .Y(N2) );
endmodule
```